

SIS 3: Build the Screens of Your App

Jetpack Compose · Focus: layout and styling · Covers Lectures 1–5

	Details
Work	Individual
Final deadline	06.10.2026
Submit	Link to your GitHub repository in the Teams assignment
Defense	In class after the deadline, 5–7 minutes per student
AI tools	Allowed (rules in section 2)

1. Why this assignment matters

AI can now write a Compose screen in a few seconds. So it is fair to ask: why should you learn this at all, and what will make you worth hiring?

The answer is that typing code is no longer the valuable part. The valuable part is everything around it:

- **Seeing structure.** Looking at a screen and knowing it is a Scaffold with a LazyColumn of Cards, each Card a Row with a Column inside. If you can't see this, you can't tell AI what to build, and you can't fix it when it builds the wrong thing.
- **Consistency.** Real apps have hundreds of screens that must look like one product. Teams do this with a design system: one theme, one set of colors, one spacing scale. AI-generated code is often inconsistent (a random color here, a random padding there). Noticing and fixing that is a real, paid skill.
- **Reviewing code you didn't write.** More and more of a developer's day is reading code, from teammates or from AI, and deciding if it is correct, clean and safe. This assignment trains that directly.
- **Explaining your decisions.** In job interviews you will be asked to explain your code and change it live. The defense at the end of this assignment is practice for exactly that moment.

In one sentence

Use AI as much as you want to go faster, but you must be the one who understands, decides and can explain. That is the skill this assignment grades.

2. AI policy

- AI tools (Claude, ChatGPT, Copilot, Gemini, etc.) are **allowed and encouraged**.
- You must **understand every line** you submit. "The AI wrote it" is not an explanation.
- Honestly describe your AI use in AI_USAGE.md (section 6). Honest AI use is **never** penalized.

- During the defense, AI is not allowed. Android Studio autocomplete and official documentation are allowed. Your syntax does not need to be perfect.

3. Part A: Design first

Before writing any code, sketch every screen of your app (on paper and take a photo, or in Figma). Then draw boxes over each sketch and label them: Scaffold, TopAppBar, Column, Row, Box, LazyColumn, LazyRow, Card, Image, Text.

- Put the sketches in a **/design** folder in your repository.
- The sketches must be your **first commit**, before any screen code.
- It's fine if your final app changes from the sketch. Just update the sketch or explain the change in the README.

Why: this is exactly how developers plan a screen. It is the one step AI cannot do for you, because it requires knowing what you want.

4. Part B: Requirements for the app

5.1 Screens and layout

- At least **3 screens**: a list screen, a detail screen, and one more of your choice (profile, settings, form, about, favourites).
- The list screen uses a **LazyColumn** with at least 10 items.
- At least one **LazyRow** (categories, tags, a horizontal gallery).
- Every screen uses **Scaffold** with a **TopAppBar**. Screens other than the start screen have a working back button.
- The layout does not break with real content: long titles are cut with an ellipsis (maxLines + overflow), and an **empty list shows a friendly empty state**, not a blank screen.
- At least one **Image** from resources with a meaningful contentDescription.

5.2 Styling and consistency

- A **custom color scheme** for your app. Tip: generate it with Material Theme Builder.
- **No hardcoded colors or font sizes inside screens**. Colors come from MaterialTheme.colorScheme, text styles from MaterialTheme.typography. Only the theme file may contain color values.
- **Consistent spacing**: use only a few values (for example 4, 8, 16, 24 dp), ideally defined once in a Spacing object.
- **Dark mode** works and looks good.
- Buttons and clickable icons are at least **48 dp** so they are easy to tap.

Self-test for styling

Change your primary color in one place. If the whole app changes, you did it right. If you have to edit several files, you still have hardcoded colors.

5.3 Structure

- At least **3 reusable composables** of your own, with parameters, each used in more than one place (for example ItemCard, SectionHeader, TagChip).
- A **@Preview** for every screen and every reusable component, including at least one dark mode preview.
- Data is defined with a Kotlin **data class** and a list.

5.4 Navigation and state

- **Navigation Compose** connects all screens.
- The detail screen **receives an id as a navigation argument** and finds the item by that id.
- At least one interactive element using **remember + mutableStateOf** (favourite toggle, filter chips, expandable card, tabs, etc.).

6. Part C: Git, README and AI log

Git and README

- Public GitHub repository.
- At least **5 meaningful commits spread over the week** (for example "Add detail screen", "Extract ItemCard component"). One big upload at the end does not count.
- README.md with: app name and short description, **screenshots of every screen in light and dark mode**, and the sketch-to-app comparison.

AI_USAGE.md

A short file (half a page is enough) in the root of your repository that answers:

- Which AI tools did you use, and for what?
- Your 3 most useful prompts (copy them).
- **One case where the AI was wrong, inconsistent or produced something ugly, and how you noticed and fixed it.** This is the most important question.
- What did you change or write by hand?
- If you did not use AI, write that, and what you used instead.

7. Defense: your points count only after it

All the points in this assignment become valid **only after you pass the defense**. The defense checks that the work is really yours, meaning that you understand it, whether or not AI helped you write it.

What happens (5–7 minutes)

- **Demo (1 min):** run the app and click through all screens.
- **Live change (2–3 min):** the teacher asks for a small change, for example "put the text under the image instead of next to it" or "add more space between the cards". You do it in Android Studio without AI. Autocomplete and documentation are allowed, and small syntax mistakes are fine.

- **Questions (2 min):** two questions about your own code, such as "how does the id get to the detail screen?" or "where do your colors come from?"

Outcomes

Result	What it means
Passed	You can explain your code and make the change. All your points count.
Partly passed	You understand most of the app but cannot explain a specific part. The points for that part are removed; the rest counts.
Not passed	You cannot explain your own code or make a basic change. The work is not accepted and receives 0 points.

How to prepare

Before the defense, open every file and ask yourself: "Could I explain this line to a classmate?" If not, ask AI to explain it to you, then try to explain it back in your own words. That is ten minutes well spent.

8. Grading

Total: 100 points + up to 10 bonus. Remember: every row counts only after the defense.

Part	What is checked	Points
A. Design	Labeled sketches of all screens, committed first	10
B1. Screens and layout	3+ screens, LazyColumn, LazyRow, Scaffold + TopAppBar, back navigation, ellipsis, empty state, Image with contentDescription	25
B2. Styling	Custom color scheme, no hardcoded colors/fonts in screens, consistent spacing, dark mode, 48 dp touch targets	25
B3. Structure	3+ reusable composables, previews (incl. dark), data class	15
B4. Navigation and state	Navigation Compose, id passed as argument, remember + mutableStateOf interaction	15
C1. Git and README	5+ meaningful commits over time, README with light/dark screenshots	5
C2. AI_USAGE.md	Honest answers, especially the "AI was wrong" case	5
Total		100

Bonus (up to +10)

These topics are not covered in lectures. They reward curiosity, and AI can help you learn them:

- Bottom navigation bar between main sections
- Layout that adapts to tablets or landscape
- A custom font from Google Fonts
- Simple animations (for example `AnimatedVisibility` for an expandable card)

9. Checklist before you submit

- ☐ /design folder with labeled sketches (first commit)
- ☐ 3+ screens, all with Scaffold + TopAppBar
- ☐ LazyColumn (10+ items) and LazyRow
- ☐ Detail screen receives an id through navigation
- ☐ Back navigation works
- ☐ Long text uses ellipsis; empty list shows an empty state
- ☐ Image with `contentDescription`
- ☐ Custom color scheme; no hardcoded colors or font sizes in screens
- ☐ Consistent spacing; clickable elements at least 48 dp
- ☐ Dark mode looks good
- ☐ 3+ reusable composables; previews for all screens and components
- ☐ At least one remember + `mutableStateOf` interaction
- ☐ 5+ meaningful commits over two weeks
- ☐ README with light and dark screenshots
- ☐ AI_USAGE.md
- ☐ I can explain every file in my project